Module 1, Lecture 1 — The Lead's First Move: Problem, Assumptions, Decisions

> **Course:** Staff Engineer / Technical Lead Mentorship **Module:** 1 — Technical lead mindset, requirements discovery, stakeholder communication, decision-making **Lecture:** 1 of Module 1 **Running example:** A personal finance application (React Native + FastAPI + PostgreSQL) **Format:** Live-lecture style. Read it like I am talking to you in a classroom.

Session tracker (I keep this updated every turn)

Item	Status
Current module / lecture	Module 1, Lecture 1
Diagnostic questions (Part 0)	**Waiting for your answers**
Hands-on exercise (Part 7)	**Waiting for your submission**
Recall test (Part 9)	**Waiting for your answers**
Unresolved questions	None yet. Yours will be listed here
Next lecture	**Not started.** I will only start it when you tell me you are ready

---

Part 0 — Before we start: let me find out where you are

Welcome. Before I teach you anything, I want to know what you already think. A good mentor does not lecture into a void. If I skip this step, I will either bore you with things you know or lose you on things you don't.

Please answer these in your own words, in a few sentences each. **Do not search the internet and do not ask an AI.** A wrong answer is far more useful to me than a perfect copied one. If you don't know, write "I don't know." That is a valid and useful answer.

1. **Role.** In your own words, what does a technical lead do that a strong senior developer does not necessarily do?
2. **Vague request.** A client messages you: *"I want a finance app."* What are the first three things you would want to find out? Why those three?
3. **Requirement vs solution.** What is the difference between a *requirement* and a *solution*? Give one example of each from any app you know.

4. **Assumptions.** What is an *assumption* in a software project? If you have an assumption you cannot verify yet, what would you do with it?
5. **Painful decisions.** Think about your DBA work. Have you seen a database design decision that was painful or expensive to change later? What was it and why was it painful?
6. **Choosing between options.** Right now, how do you decide between two technical options, for example PostgreSQL vs MongoDB, or React Native vs a web app?
7. **Documents.** Have you ever written or read a project document (requirements, design, decision record)? What was it, and was it useful?
8. **Self-rating.** On a scale of 1 to 5, rate yourself on: (a) writing code, (b) designing databases, (c) talking to non-technical people about scope, (d) estimating how long work will take.

What I will do with your answers: I will compare them with what you read below, point out where your instincts are already strong, and where the gaps are. I'll adapt the pace of later lectures to match.

You can read the lecture first and answer after, but I would rather you answer first. Your "before" answers are valuable because we will compare them with your "after" answers at the end of the lecture.

Part 1 — Learning objective

By the end of this lecture, you should be able to:

1. **Explain** why a technical lead's first job is not choosing technology, but reducing uncertainty.
2. **Separate** a client's request into: goal, problem, requirement, and proposed solution.
3. **Classify** any statement in a project as a *known fact*, an *assumption*, or an *unknown*.
4. **Write** a short problem statement and an assumption log for a vague request.
5. **Classify** technical decisions as *easy to reverse* or *hard to reverse*, and decide how much care each deserves.
6. **Reply** to a client who wants a price and timeline immediately, without lying and without saying "it depends" and going silent.

Notice the verbs: explain, separate, classify, write, reply. This is a skill lecture. By the end you should be able to *do* these things, not merely recognise the vocabulary.

Part 2 — Real-world scenario

Imagine you are a freelancer. You get this message on a Monday morning:

> **Meera:** *"Hi! I want to build an app like those money apps, but simpler. My friends keep asking me for something like that. Can you tell me how long it will take and what it will cost? I'd like to launch in 2 months."*

Think about what you feel when you read that.

If you are like most developers early in their career, your brain does something like this:

- "Okay, money app. Transactions, categories, charts."
- "React Native for mobile, FastAPI for backend, Postgres for the database."
- "Maybe 8 weeks. Let me say 2 months and a price."
- Opens the editor. Creates a `transactions` table.

That reaction is natural. It is also how many freelance projects go wrong.

Look at what is actually in that message:

What Meera said	What is hiding inside it
"An app like those money apps"	Which apps? Which features of them? Budgeting? Expense tracking? Investing? Bank syncing?
"But simpler"	Simpler how? Fewer features? Easier screens? Less setup?
"My friends keep asking me"	Is this a real market, or a feeling? Who are "friends"? Will they pay?
"How long and how much?"	Based on what? She has not told you what to build.
"Launch in 2 months"	Launch to whom? Public app stores? Ten friends? Is there a hard date, or a hope?

One message, five hidden questions, and zero requirements. An experienced lead sees this immediately. That is the skill we are building today.

Part 3 — The lead's thought process

I'm going to walk you through how an experienced lead thinks. I will go in six steps. Take your time. This is the core of the lecture.

3.1 First: what is a technical lead actually responsible for?

Many people think a technical lead is "the best coder on the team." That is not the definition. A strong lead is often a strong coder, but the *job* is different.

A technical lead is responsible for **outcomes**, not **output**.

- **Output** is code written, tickets closed, screens built.
- **Outcome** is: did the user's problem actually get solved, safely, on time, at a cost the business can sustain?

You can produce a lot of output and zero outcome. Imagine building a beautiful finance app that nobody uses because it asks for 14 fields to log one expense. Lots of output, no outcome.

I like to describe the lead's job as four responsibilities:

1. **Clarity.** Everyone knows what is being built and why.
2. **Decisions.** The important choices are made deliberately, by the right people, at the right time, and recorded.
3. **Risk.** The things that could go wrong are found early, while they are cheap.
4. **Communication.** Stakeholders (clients, users, teammates) are never surprised.

Notice that none of these is "write the code." Code matters, but code is the *consequence* of the four above.

DBA connection. You already do this. When a developer asks you "can you add an index?", a good DBA does not just run `CREATE INDEX`. You ask: which query is slow? How often does it run? How big is the table? What is the write overhead? Is the real problem the index, or the query? You are already practicing "problem before solution" on the database side. Today we apply the same instinct to the whole project.

3.2 Second: separate goal, problem, requirement, and solution

This is the most important mental tool in the lecture. People mix these four things up constantly, and the mix-up causes most scope trouble.

Here are the four levels, from "why" down to "how":

```
GOAL         Why does this matter to the person?
   ↓
PROBLEM      What is getting in the way right now?
   ↓
REQUIREMENT  What must be true for the problem to be solved?
   ↓
SOLUTION     What specific thing do we build to make it true?
```

Let me use a non-software example first, so it sticks.

- **Goal:** "I want to get to work comfortably."

- **Problem:** "My commute takes 90 minutes and the train is crowded."
- **Requirement:** "I need a way to get to work in under 45 minutes with a guaranteed seat."
- **Solution:** "Buy a motorcycle." *(Or move closer. Or negotiate remote work. Or take a different train line.)*

See what happened? If I only listen to "buy a motorcycle," I never discover that remote work two days a week would be cheaper and safer. **A solution is only one of many ways to satisfy a requirement.** If you accept the solution too early, you lose the chance to find a better one.

Now apply it to Meera:

- **Goal (guess):** "I want to feel in control of my money." *(I do not know this yet. It is a guess, and I must verify it.)*
- **Problem (guess):** "I don't know where my money goes each month."
- **Requirement (guess):** "I must be able to see my spending grouped by category for a month."
- **Solution (what she half-said):** "An app like those money apps."

Notice something: *I am labelling every one of these as a guess.* That is a habit I want you to build. **A lead says out loud which parts are verified and which are guesses.**

What an inexperienced developer misses: they start at the bottom of the ladder (solution) because it is the most concrete and the most comfortable. Technology feels like progress.

What an experienced lead investigates: they climb *up* the ladder with questions: "Why do you want this?" "What do you do today?" "What would success look like in three months?" The lead climbs up to the goal, then comes back down with options.

A very light version of this is called the **"Five Whys"** (a technique from manufacturing, popularised by Toyota). You keep asking "why" until you reach the real cause. Do not literally ask five times in a meeting, because it sounds like an interrogation. Use softer forms: "Help me understand what happens today when you…", "What would be different if this existed?"

3.3 Third: the questions a lead asks

Now let me show you how a lead organises questions. Rather than asking randomly, I group them into six categories. You can use this as a mental checklist (not a script to read aloud).

Category	What I am trying to learn	Example questions for Meera
Who	Who are the users and who decides/pays?	Who will use it first? Just you and friends, or the public? Who pays for development?
Why	The goal and problem behind the request	What do you do today to track money? What goes wrong? Why now?
What	The smallest thing that would be valuable	If the app could do only one thing on day one, what would it be?
Constraints	Time, money, people, rules	Is "2 months" a hard date? Is there a budget ceiling? Will you be available for feedback weekly?
Risk	What could hurt us	Will the app hold real financial data? Does it connect to banks? Are there legal requirements in your country?
Success	How we will know it worked	After launch, what number would make you say "this worked"? 50 users? 5?

Why does this order matter? Because **Who and Why come before What.** If you ask "what features do you want?" first, you get a wish-list. If you ask "who is this for and what hurts today?" first, you get *reasons*, and reasons let you design.

One more important point: you will not get every answer. Clients often do not know. That is not a failure; it is information. "I don't know" from a client means *an assumption is needed here, and it is risky.* We will track that.

3.4 Fourth: known facts, assumptions, and unknowns

Every project statement is one of three things:

1. **Known fact.** Verified. Someone with authority confirmed it, or you saw it directly. *"The client wants an Android and iOS app" (she said so in writing).*
2. **Assumption.** Something we are *treating as true* so we can move forward, but we have not verified. *"Users will log expenses manually."*
3. **Unknown.** We know we do not know. *"Does the app need to connect to bank accounts?"*

Here is the part most people miss: **the dangerous one is the assumption, not the unknown.** An unknown is obvious. You will ask about it. An assumption is *invisible*. It feels like a fact, so nobody checks it. Projects rarely fail because of what the team knew they did not know. They fail because of what the team *believed* without checking.

A lead makes assumptions visible by writing them down in an **assumption log**. For each assumption, I record:

- **The statement.** What are we assuming?
- **Why it matters.** What design/plan depends on it?
- **Cost if wrong.** How painful is it if this turns out false?
- **How to validate.** Who can confirm, and how?
- **Owner and status.** Who is responsible? Open, confirmed, or rejected?

The "cost if wrong" column is where the lead's judgment lives. Not all assumptions deserve equal attention. An assumption with *high impact if wrong* and *cheap to check* should be checked today. An assumption with *low impact if wrong* can wait or be ignored.

You can think of it as a simple two-by-two:

```
                    Cost if wrong
                 LOW            HIGH
            +-------------+--------------+
Cheap to    |  Just check |  CHECK NOW   |
verify      |  when easy  |  (top priority)
            +-------------+--------------+
Expensive   |  Ignore /   |  Plan a      |
to verify   |  accept     |  spike or    |
            |             |  prototype   |
            +-------------+--------------+
```

> A **spike** is a small, time-boxed experiment to learn something. "Spend one day trying to import a bank statement CSV and see how messy the data is." We will study spikes properly later.

DBA connection. This is capacity planning. You never say "the database will be fine." You say, "I'm assuming 10,000 rows per day. If it is actually 1 million, the partitioning plan changes." A DBA who writes down growth assumptions and revisits them is already thinking like a lead.

3.5 Fifth: not all decisions are equal (reversible vs hard to reverse)

This is the idea that separates juniors from seniors in practice. Let me say it plainly:

> **Decide quickly on things that are cheap to change. Decide slowly and carefully on things that are expensive to change.**

Some people call these "two-way doors" and "one-way doors." A two-way door: if you walk through and dislike the room, you walk back out. A one-way door: once you are through, you are stuck.

Examples in our finance app:

Decision	Door type	Why
Button colour, screen label text	Two-way	Change in minutes. No data impact.
Which charting library to use	Mostly two-way	Replaceable in days. Annoying, not catastrophic.
Naming of an API endpoint (before any client uses it)	Two-way	Free to change before release.
Naming of an API endpoint (after mobile apps are installed on thousands of phones)	**One-way-ish**	Old app versions in the wild will still call it. Now you need versioning.
How money is stored (float vs exact decimal vs integer minor units)	**One-way**	Data is already stored and calculated. Fixing it means migrating every row and every calculation.
Whether data belongs to a single user or to a shared household	**One-way**	It changes ownership in almost every table and every permission check.
Whether you store timestamps in UTC with time zone awareness	**One-way-ish**	Wrong historical timestamps may be impossible to reconstruct.
Primary key strategy (sequential integer vs UUID)	**Hard to reverse**	Every foreign key, index, and external reference depends on it.

Notice something about the one-way decisions: **almost all of them are about data.** That is good news for you. You are a DBA. You already understand, in your bones, that *data outlives code.* You can rewrite an application in a new language in a few months. You cannot easily rewrite five years of financial records that are wrong.

This is your hidden advantage as you grow into a lead. Many pure application developers underestimate how hard schema decisions are to undo. You do not. Use that instinct deliberately.

There is a related idea called the **last responsible moment**: delay a decision until the latest time at which delaying would cause harm, because you learn more over time. But careful: this applies to *two-way* decisions. For one-way doors that are certain to be needed, you cannot postpone them to "later." The schema must exist before the first row is written.

Established practice vs context-dependent:

- *Established practice:* classify decisions by reversibility and spend effort accordingly. Nearly every serious engineering organisation does this in some form.
- *Context-dependent:* where a specific decision falls. For a throwaway prototype for three friends, storing money as a float is a cheap two-way door because you will throw it away. For a production app, it is a one-way door. **Context changes the classification.**

3.6 Sixth: talking to stakeholders about uncertainty

A stakeholder is anyone who has a stake in the outcome: client, user, manager, teammate. Here is the communication trap:

- Client: "How long will it take?"
- Weak lead: "Two months." *(A guess dressed as a promise.)* **or** "It depends." *(True, but useless. It sounds like you're hiding.)*

A strong lead does a third thing. They give a **structured answer** with three parts:

1. **What I know.** "You want a mobile app for tracking spending."
2. **What I'm assuming, or don't yet know.** "I don't yet know whether it needs bank connections, and that single feature could double the effort."
3. **What I need from you and when I'll come back.** "Can we do a 45-minute call this week? After that I can give you a range with the assumptions written down."

Then, when you do give an estimate, you give a **range tied to assumptions** ("6 to 9 weeks, assuming manual entry and no bank sync"), not a single number. We will cover estimation deeply in Module 4. For today, remember the principle: **an estimate without assumptions is just a wish.**

The tone matters too. You are not being difficult. You are being *professional*. Clients generally respect a freelancer who asks sharp questions more than one who says yes to everything. The one who says yes to everything is the one who sends the "sorry, we need more time" email in week 7.

Part 4 — Practical demonstration: Meera's request, step by step

Let's now work through Meera's message the way a lead would. I will show a **weak approach** first, then a **stronger approach**.

> **Important note:** The "answers" Meera gives below are *invented for teaching*. In your real projects, you must never invent requirements. You write down only what the stakeholder actually said, and mark everything else as an assumption.

4.1 The weak approach

```
Meera: "...How long will it take and what will it cost? Launch in 2 months."

Developer: "Sure! Around 2 months, $X. I'll use React Native, FastAPI and
            PostgreSQL. I'll start on the database design tonight."
```

Then the developer creates this:

```sql
CREATE TABLE transactions (
    id SERIAL PRIMARY KEY,
    amount FLOAT,
    category TEXT,
    note TEXT,
    created_at TIMESTAMP
);
```

Let me show you the problems with this, one at a time, because each problem is an example of what happens when decisions are made before understanding:

1. **No user column.** Is this a single-user app? The developer assumed so silently. It is an invisible assumption.
2. `FLOAT` **for money.** This is a well-known mistake. Floating point cannot represent many decimal values exactly. You can see it yourself in PostgreSQL:

```sql
SELECT 0.1::float8 + 0.2::float8 = 0.3::float8;   -- returns false
SELECT 0.1::numeric + 0.2::numeric = 0.3::numeric; -- returns true
```

Tiny errors accumulate across thousands of transactions, and a finance app that disagrees with a bank statement by a few cents destroys trust. 3. **No currency.** Silent assumption: one currency forever. 4. `TIMESTAMP` **without time zone.** Silent assumption: all users are in one place. 5. **No amount sign convention.** Is spending negative or positive? Silent assumption again. 6. **Commitment to a timeline and price** before knowing whether bank sync is in scope.

Every one of these is an *assumption that was made without anyone deciding to make it.* That is the real failure: not the wrong column type, but the *unconscious decision.*

4.2 The stronger approach: step by step

Step 1 — Restate the request in your own words. I write this for myself first, then I confirm it with the client.

> "Meera wants a mobile app that helps people understand their spending. She sees demand from friends. She wants to launch in about 2 months. She has not specified features, platforms, or budget."

Why restate? Because it forces me to notice what is *missing*. If I cannot restate it, I do not understand it.

Step 2 — Sort every statement into fact / assumption / unknown.

Statement	Type	Notes
Meera wants a money app	Fact	She said it.
"Simpler than existing apps"	Unknown	Simpler in what way?
Friends want it	Assumption	Evidence is "my friends keep asking." Unverified demand.
Launch in 2 months	Unknown	Hard date or wish?
Users will type expenses manually	Assumption	Not stated by Meera. I made it up.
App needs bank connection	Unknown	Huge effort impact.
Single currency	Assumption	Not stated.

Look at the last three rows. Before this step, I would have treated "manual entry" and "single currency" as obvious. After this step, I see I invented them.

Step 3 — Prioritise the questions. I do not ask 40 questions. I ask the 5 or 6 that change the plan the most. I rank questions by: *"If the answer is A versus B, does my plan change dramatically?"*

1. Does the app need to connect to bank accounts, or will users enter expenses themselves? *(Changes effort, cost, security, and legal exposure massively.)*
2. Who are the first users, and how many? *(Changes whether it is a prototype or a production product.)*
3. What is the single most important thing the app must do on day one? *(Defines the minimum viable product, which is Module 2.)*

4. Is the 2-month date fixed, and why? *(Tells me whether to cut scope or move the date when conflict arrives.)*
5. Is there a budget range? *(Constrains the solution.)*
6. Will real financial data be stored? In which country? *(Security and regulation.)*

Step 4 — Hold the discovery call. Here is the simulated conversation (invented for teaching):

> **Me:** What do you do today to track money? **Meera:** I use a notebook. I forget to write things down and then at month-end I have no idea.
> **Me:** If the app only did one thing on day one, what would it be? **Meera:** Let me add an expense in a few seconds and see how much I spent on food, travel, and so on this month.
> **Me:** Do you want it to connect to bank accounts? **Meera:** Oh, that would be amazing, but I guess not at first.
> **Me:** Who would use it at launch? **Meera:** Me, and maybe 10 friends to test.
> **Me:** Is the 2-month date fixed? **Meera:** I told my friends "in a couple of months." Not fixed.

Notice what I learned. **The goal is not "a finance app." The goal is "stop forgetting expenses and see where money goes."** The problem is *effortful, forgotten logging*. That changes design: speed of entry becomes the most important quality of the product. A product with 40 features but a 30-second add-expense flow would fail Meera.

Also notice: "I guess not at first" about bank sync is a **soft** answer. I must record it as *"deferred, not rejected"* because it may come back and it affects architecture.

Step 5 — Write the problem statement.

> "Meera, and a small group of friends, forget to record daily expenses in a notebook and cannot see their monthly spending by category. We will build a mobile app that makes recording an expense take only a few seconds and shows monthly spending by category. Success at launch is that the first 10 users record expenses in at least 3 weeks out of the first 4."

Let me explain why each part exists:

- *Who:* Meera and ~10 friends.
- *Pain:* forgetting, no month-end visibility.
- *Solution direction:* fast logging plus category summary. (It is a direction, not a final design.)
- *Success measure:* concrete and testable. *(Note: I proposed this number. Meera must confirm it. I mark it as a proposal.)*

Step 6 — Write the assumption log.

#	Assumption	Why it matters	Cost if wrong	How to validate	Owner	Status
A1	Users enter expenses manually; no bank sync in v1	Drives architecture, security scope, cost	**High**: bank sync can double effort and adds compliance work	Confirm in writing with Meera	Me → Meera	Open (verbal "not at first")
A2	Single currency (INR) in v1	Affects money schema	**High**: adding currency later requires backfilling every row and revisiting every calculation	Ask Meera whether friends travel/abroad	Me → Meera	Open
A3	Each user sees only their own data (no shared households)	Affects every table and permission check	**High**: sharing later touches almost the whole schema	Ask: "Will couples or families share a budget?"	Me → Meera	Open
A4	~10 users at launch, low traffic	Affects hosting and performance planning	Low at first	Meera's estimate	Meera	Confirmed (verbal)
A5	2-month launch is flexible	Affects scope-vs-date trade-off	Medium	Meera said "not fixed." Ask once more for firm confirmation	Me → Meera	Mostly confirmed

Look at the "cost if wrong" column. A1, A2, and A3 are the dangerous ones. They are *cheap to ask* (one sentence to Meera) and *very expensive if wrong*. Those go to the top of the list. Using the two-by-two from earlier, they are "CHECK NOW."

Step 7 — Classify the early decisions by door type.

Decision	Door type	What the lead does
Money stored as integer minor units (e.g., paise) or `NUMERIC`, never float	One-way	Decide now and record it.
Currency stored per transaction, even though only INR is used in v1	One-way, but **cheap insurance**	One extra column now (`currency_code`) avoids a painful migration later.
`user_id` on every user-owned table	One-way	Decide now. Even if single-user today, adding ownership later is painful.
UTC timestamps with `TIMESTAMPTZ`	One-way-ish	Decide now.
Charting library	Two-way	Do not spend a meeting on this.
Screen layout and colours	Two-way	Iterate with real users.

Now you see the pattern: for the one-way decisions I add **cheap insurance**. Adding one `currency_code` column costs 30 seconds today. Adding it after launch costs a migration, a backfill, and a risk of mistakes. This is the DBA mindset applied to project planning: *protect the data model, stay flexible everywhere else.*

Here is what the improved first draft of the table might look like, **not as a finished design** (that is Module 3), but to show how the decisions above leave fingerprints:

```sql
CREATE TABLE transactions (
    id              BIGSERIAL PRIMARY KEY,
    user_id         BIGINT NOT NULL,                -- A3: ownership from day one
    amount_minor    BIGINT NOT NULL,                -- integer minor units (paise)
    currency_code   CHAR(3) NOT NULL DEFAULT 'INR', -- A2: cheap insurance
    category_id     BIGINT,                         -- details to be designed late
    note            TEXT,
    occurred_at     TIMESTAMPTZ NOT NULL,           -- time zone aware
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

I want you to notice something important: **I did not "design the database" here.** I only let the decisions I had already made show through. The table is a *consequence* of the thinking, not the thinking itself. If I handed you only this table, without the log of why, you would not know that

`currency_code` exists because of assumption A2. A month later even *I* would forget. That is why documentation of *reasoning* matters (Part 5).

Step 8 — Reply to Meera.

> "Thanks Meera, this is a great idea and I'd love to help. From our call, here is what I understand: the app should let you record an expense in a few seconds and show your monthly spending by category. For the first version it's you and about 10 friends, with no bank connection. I've written down three assumptions I want to confirm: no bank sync in version 1, one currency (INR), and each person sees only their own data. If those hold, my first estimate is a range of X to Y weeks, and I'll send it with the assumptions by Thursday. If bank sync or shared budgets are needed, the range changes, and I'll tell you how much. Could you confirm those three points by reply?"

Check it against the three-part structure: what I know, what I assume, what I need from you. And it contains a **commitment to a next action with a date**, which builds trust.

Part 5 — Documentation: what the lead records, and how much

I want to be careful here. This course is about *thinking*, and documents are only a way to store thinking. A lead does not write documents to look professional. A lead writes documents because **memory fails, people leave, and clients forget what they agreed.**

For this lecture, a lead records four small things. Together they fit on one or two pages. That is intentional.

5.1 Discovery notes

- **What it is:** a plain record of what stakeholders actually said, with dates.
- **Why it matters:** when a dispute arises ("I never asked for that!"), you can point to what was said.
- **Level of detail:** bullet points, in the stakeholder's own words where possible. Not polished.

```
Discovery call — Meera — 2 sessions, 45 min
Said: uses a notebook today; forgets to record; wants quick add-expense
Said: "bank connection would be amazing, but not at first"
Said: launch date is "not fixed"
Not discussed: budget, shared budgets, currency
```

Notice "Not discussed." Listing what was *not* covered is as valuable as what was.

5.2 Problem statement

- **What it is:** two to four sentences: who, pain, direction, success measure.
- **Why it matters:** it's the compass. When someone proposes a feature, ask: "Does this serve the problem statement?"
- **Level of detail:** short enough to remember. If it needs a page, you do not yet understand the problem.

5.3 Assumption and open-questions log

- **What it is:** the table from Step 6.
- **Why it matters:** it converts invisible risk into visible, trackable work.
- **Level of detail:** one row per assumption that would change the plan if false. Do not list trivial ones. A log with 80 rows will not be read.
- **Habit:** review it at the start of every planning session. Move rows from Open to Confirmed or Rejected. Add new ones.

5.4 Decision log (a light "Architecture Decision Record")

An **Architecture Decision Record (ADR)** is a short document capturing one significant decision: the context, the options considered, the choice, and the consequences. We will study ADRs more formally in Module 3. For now, use a lightweight form:

```
DECISION D-001: Store money as integer minor units (BIGINT) with currency_code
Date: 2026-10-12      Status: Accepted      Decided by: Me (with Meera informed)

Context:   Finance app; amounts must match bank statements exactly.
Options:   (a) FLOAT — rejected: cannot represent decimals exactly
           (b) NUMERIC(12,2) — acceptable; exact; scale fixed at 2
           (c) BIGINT minor units — chosen; exact; fast; forces explicit
               currency handling
Decision:  (c), with currency_code CHAR(3) on every amount.
Consequences: Must convert at the UI boundary (divide by 100 to display).
              Currencies with different minor units (e.g. JPY has 0) need care.
Revisit if: We add multi-currency conversion or crypto.
```

Notice "**Revisit if.**" It records *the condition under which the decision might change.* This is what makes it a professional record and not a stone tablet. Remember, I told you earlier: **do not claim one universally correct process.** Option (b) `NUMERIC` is also perfectly defensible. Many teams prefer it for readability. The point of the log is not that I'm right; it is that the reasoning is visible and challengeable.

5.5 Anti-patterns (what not to do)

- **The 40-page requirements document nobody reads.** Too much detail before learning anything.
- **Documents written after the fact to justify decisions.** They record conclusions without reasoning.
- **Assumptions hidden inside designs.** If an assumption only exists in your head or inside a table definition, it is a landmine.
- **No dates, no owners.** A log without owners is a wish list.

Part 6 — Trade-offs and failure scenarios

A good lead is never religious about a technique. Let me show you where this approach can fail and what alternatives exist.

6.1 Too little discovery

Scenario: The developer from our weak example builds for 7 weeks. At the demo, Meera says, "Oh, my husband and I want to share one budget." The data model assumed one user per transaction. Ownership touches every table, every API permission check, and every screen. The rewrite costs three weeks.

Cost: money, trust, deadline. And most painfully, it was a *one-minute question* at the beginning.

6.2 Too much discovery (analysis paralysis)

Scenario: A lead insists on 12 meetings, a 30-page document, and a complete assumption log before writing a line of code. Meanwhile the client's enthusiasm fades, and a competitor ships something.

The balance: time-box discovery. For a small project, perhaps two focused conversations and a one-to-two-page summary. For a regulated banking product, much more. **The depth of discovery should scale with the cost of being wrong.** This is context-dependent. There is no universal number.

6.3 The stakeholder who "doesn't know"

Sometimes the client cannot answer. Options:

- **Make a clearly labelled assumption** and proceed, noting the risk.
- **Run a spike or prototype** to learn cheaply. (Show Meera a clickable mock of the add-expense screen and watch her reaction.)
- **Design for flexibility at the one-way doors** (like adding `currency_code`) while staying simple elsewhere.

- **Contract for it.** In freelance work, put "changes to these assumptions will change the estimate" in writing.

6.4 Fixed-price vs time-and-materials (a freelance trade-off)

- **Fixed price:** the client likes certainty. But *you* absorb the risk of wrong assumptions. Only do this with a well-understood scope, with assumptions written into the agreement.
- **Time and materials:** you bill for time spent. The client carries more risk, and some clients dislike open-ended cost. Works well when scope is uncertain.
- **Hybrid:** a fixed-price discovery/design phase first, then re-estimate. Many freelancers use this because it prices the *learning* separately from the *building*.

This is context-dependent. No option is always right. What matters is that you choose consciously, and that your assumptions are in writing.

6.5 Over-engineering the one-way doors

If you take today's lecture too far, you might say, "I must make every decision perfectly right now!" That is wrong in the other direction. Adding `currency_code` is cheap insurance. Building a full multi-currency conversion engine "just in case" is over-engineering: it is expensive, and you may never need it. The rule is **cheap insurance, not speculative construction.** Protect the *shape* of the data; do not build features nobody asked for.

6.6 Resume-driven development

A subtle failure: choosing technology because *you* want to learn it, not because the problem needs it. You are currently learning React Native, FastAPI, and TypeScript, and that is a perfectly fine reason to choose them for your *own* learning project. But for a *client's* production project, the stack choice should be justified by the client's needs, team skills, and maintenance cost. Be honest with yourself about which situation you are in. If you do choose a stack partly to learn, tell the client, and consider the risk.

Part 7 — Hands-on exercise

Now it is your turn. Take your time. Write the answers in your own words. Quality over speed. **Please do not ask for hints before you have made a real attempt.** If you get stuck, tell me where you are stuck, and I will give you a hint, not the answer.

The scenario

You are the freelance lead for the same personal finance app. A different stakeholder, **Rahul**, who is a friend of Meera and wants to be the second paying user of your product (invented for teaching), sends you this message:

> **Rahul:** *"I really like the idea. For my version I want the app to automatically pull my bank transactions so I never have to type anything. It should also work offline when I'm on the*

> *metro. And can you add this in the first release? Also my wife and I want to see our spending together."*

Tasks

Task 1 — Restate and separate (10 minutes). Write one paragraph restating Rahul's request in your own words. Then fill in this ladder, **marking each line as "verified" or "my guess":**

- Goal:
- Problem:
- Requirements (at least three):
- Solutions he has proposed (identify at least two statements that are *solutions*, not requirements):

Task 2 — Questions (10 minutes). Write **8 to 10 questions** you would ask Rahul, grouped by the six categories (Who, Why, What, Constraints, Risk, Success). Then pick your **top three** and explain *why* each one would change your plan the most.

Task 3 — Assumption log (10 minutes). Write **at least four** assumption log entries, using the columns: Assumption, Why it matters, Cost if wrong, How to validate, Status. Include at least one that is "high cost if wrong, cheap to verify."

Task 4 — Decisions and doors (10 minutes). List **at least five** technical or product decisions that Rahul's message forces you to think about. Classify each as a one-way or two-way door, and explain in a sentence why. Include at least one *database-related* decision, and say what a DBA would worry about.

Task 5 — Reply (10 minutes). Write a reply to Rahul of **no more than 150 words**, following the structure: *what I understand → what I'm assuming or don't know → what I need from you and when I'll respond.* The reply must not promise a date or price you cannot justify.

Task 6 — One-paragraph reflection. What was the hardest part? What did you want to do that this lecture told you not to do?

What I'll look for (so you know the standard)

- Do you separate *solution* statements from *requirements*?
- Do your questions climb toward the "why," or just collect features?
- Are your assumptions *specific*, or vague like "the app should work well"?
- Did you notice that three things in Rahul's message are each large, independent, expensive features (bank sync, offline mode, shared accounts)? How did you handle that?
- Is your reply honest and useful at the same time?

Part 8 — Review (this part happens after you answer)

I cannot evaluate your answer yet because you have not given it. This is how the review will work, so there are no surprises:

1. **What you did well.** I will name specific strengths, not generic praise.
2. **Gaps.** I will point out what is missing or weak, and *why it matters*, not only that it is wrong.
3. **Hints before answers.** For gaps, I will give hints first and ask you to revise. I will reveal the full solution only if you are still stuck after trying.
4. **Comparison with your Part 0 answers.** To show you how your thinking has changed.
5. **A revise-and-resubmit request.** Almost always, I will ask you to improve one or two specific items. This is deliberate. Revision is where learning happens, and it is also how real code reviews work.

> **Status:** Waiting for your Part 0 answers and Part 7 submission.

Part 9 — Recall test

Do these **without looking back** at the lecture. If you cannot answer without looking, that is useful information; tell me which ones.

1. In two or three sentences: why is a technical lead responsible for *outcomes* rather than *output*? Give an example from the finance app of high output and no outcome.
2. Explain the ladder (goal → problem → requirement → solution) using an example that is **not** from this lecture.
3. Why is an **assumption** more dangerous than an **unknown**?
4. A decision is "high cost if wrong, cheap to verify." What do you do with it, and when?
5. Name three decisions in a finance app that are hard to reverse, and explain why *data* is involved in each.
6. Why did we add a `currency_code` column even though v1 only uses INR? Why is that *not* over-engineering? Where would it *become* over-engineering?
7. A client asks "how long will it take?" on day one. What are the three parts of a good answer?
8. **Challenge question:** I said "decide quickly on two-way doors and carefully on one-way doors." Describe a situation where something that looks like a two-way door is actually a one-way door. *(Hint: think about what happens when something is already in users' hands.)*

Part 10 — Completion checklist

Tick these honestly. If you cannot tick one, that is where we go back and practise.

- ☐ I can explain the difference between **output** and **outcome** with an example.
- ☐ I can take any vague request and separate **goal, problem, requirement, and solution**.
- ☐ I can sort a statement into **fact, assumption, or unknown**.
- ☐ I can write an **assumption log** entry including *cost if wrong* and *how to validate*.
- ☐ I can prioritise questions by *"does the answer change the plan?"*
- ☐ I can classify decisions as **one-way** or **two-way** and explain why data decisions usually fall in the first group.
- ☐ I can explain why **cheap insurance** differs from **speculative construction**.
- ☐ I can write a client reply with: *what I know → what I assume → what I need and when.*
- ☐ I can explain why I should not promise a single-number estimate without stating assumptions.
- ☐ I understand that these techniques are **context-dependent**: depth of discovery scales with the cost of being wrong.

Before you proceed: exactly what you should do

1. **Answer the Part 0 diagnostic questions** (8 short answers).
2. **Complete the Part 7 exercise** (Tasks 1 to 6).
3. **Attempt the Part 9 recall test** (all 8 questions, no notes).
4. **Tell me which checklist items you could not tick.**
5. Mention **any question about this lecture** you want me to add to the "unresolved questions" tracker.

Please send all of these in a single reply. I will then review your work, give hints where needed, and ask you to revise. I will **not** start Module 1, Lecture 2 until you tell me you are ready and we've closed this lecture.

When you are ready, reply with your answers. Take your time. Thinking carefully here will save you a lot of pain on real projects.